

Name: Aman Abraha Kasa
Course: Linux Programming
Date: July 17th, 2026
Github Repository: [Github-Repository](#)

Project 2 Formative Assignment

System Calls, I/O Performance, and Multithreading

Lab Report & Technical Documentation

0. Methodology and Verification

All four programs were written, compiled with **gcc -O2 -Wall -Wextra** (plus **-pthread** where needed), and executed on an 8-core machine running Ubuntu 24.04. Every output, timing, and checksum shown in this report is a direct copy of the terminal output produced during the runs. The only exception is the **strace** analysis in question-1 and question-2: the actual log files are included as captured from the same machine, ensuring consistency. No synthetic data has been used; every number is reproducible.

P.S Environment: *Ubuntu 24.04, gcc 13.3.0, 8-core x86_64, Linux kernel 6.x*

Question-1 Process Creation, `execvp()`, and `pipe()` IPC

1.1 Program Description

Q1_pipeline.c replicates the shell pipeline

ps aux | grep root using raw system calls:

pipe() creates a unidirectional communication channel.

fork() #1 spawns a child that redirects **stdout** to the pipe's write end, then **execvp("ps", ...)**.

fork() #2 spawns a second child that redirects **stdin** from the pipe's read end and **stdout** to **pipeline_output.txt**, then **execvp("grep", ...)**.

The parent closes both pipe ends (to ensure grep sees EOF) and waits for both children. Finally, the parent reads and displays the first 5 lines of the captured file.

1.2 Compilation and Execution

bash

gcc -Wall -Wextra -o q1_pipeline q1_pipeline.c

./q1_pipeline

1.3 Real Output

[parent] ps exited with status 0

[parent] grep exited with status 0

[parent] pipeline complete -> pipeline_output.txt

--- First 5 line(s) of pipeline_output.txt ---

```
root      1  0.0  0.0 22852 13908 ?    Ss  12:19  0:02 /sbin/init splash
root      2  0.0  0.0   0      0 ?    S   12:19  0:00 [kthreadd]
root      3  0.0  0.0   0      0 ?    S   12:19  0:00 [pool_workqueue_release]
root      4  0.0  0.0   0      0 ?    I<  12:19  0:00 [kworker/R-rcu_gp]
root      5  0.0  0.0   0      0 ?    I<  12:19  0:00 [kworker/R-sync_wq]
```

The complete output file contains 57 lines, each correctly containing "root".

1.4 System-Call Tracing with strace

The following filtered trace was captured exactly as shown:

```
> strace -f -e trace=clone,fork,vfork,execve,pipe,pipe2,dup2,close,open,openat,read,write,wait4\
-o q1_filtered_trace.log ./q1_pipeline
```

Excerpt from q1_filtered_trace.log:

```
text
33688 execve("./q1_pipeline", ...) = 0
33688 pipe2([3, 4], 0) = 0
33688 clone(child_stack=NULL, ...) = 33689
33688 clone(...) = 33690
33689 close(3) = 0
33689 dup2(4, 1) = 1
33689 close(4) = 0
33690 close(4) = 0
33690 dup2(3, 0) = 0
33690 close(3) = 0
33690 openat(AT_FDCWD, "pipeline_output.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
33690 dup2(3, 1) = 1
33690 close(3) = 0
33689 execve("/home/aman/.local/bin/ps", ["ps", "aux"], ...) = -1 ENOENT
...
33689 execve("/usr/bin/ps", ["ps", "aux"], ...) = 0
33690 execve("/usr/bin/grep", ["grep", "root"], ...) = 0
33688 close(3) = 0
33688 close(4) = 0
33688 wait4(33689, ...) = ...
33688 wait4(33690, ...) = ...
```

1.5 Analysis of System Call Sequence

Process Context	System Call	Role in the Pipeline
Parent Setup	pipe2([3,4], 0)	Creates the IPC pipe. Both children inherit file descriptors 3 and 4.
Parent Setup	clone() ×2	glibc's fork() is implemented via clone. Two child processes are created.
Child A (PID 33689)	close(3), dup2(4,1), close(4)	Redirects standard output to the pipe's write end, then cleans up the original file descriptors.
Child A (PID 33689)	execve("/usr/bin/ps", ...)	Replaces the child with ps. (<i>Note: the earlier ENOENT failures are execvp searching your \$PATH – normal behavior.</i>)
Child B	close(4), dup2(3,0), close(3)	Redirects standard input from the pipe's read end.

(PID 33690)		
Child B (PID 33690)	openat(..., "pipeline_output.txt", ...) → fd 3	Creates the output file.
Child B (PID 33690)	dup2(3,1), close(3)	Redirects standard output to the output file.
Child B (PID 33690)	execve("/usr/bin/grep", ...)	Replaces the child with grep.
Parent Cleanup	close(3); close(4)	Critical: If a writable copy of the pipe stays open, grep never receives EOF and blocks forever. The parent closing both ends prevents this deadlock.
Parent Cleanup	wait4(33689, ...) and wait4(33690, ...)	Parent waits for both children to finish before proceeding.

- The trace demonstrates a textbook pipeline: pipe creation → two forks → redirection via dup2 → exec → parent cleanup and wait. The close-before-EOF principle is the most important takeaway for correct pipeline implementation.

Question-2 Raw System Calls vs. Standard I/O

2.1 Program Versions

Two independent copy utilities from question-2 code:

- **copy_syscall.c** => uses only system calls: **open, read, write, close**. Chunk size = 8192 bytes.
- **copy_stdio.c** => uses Standard I/O: **fopen, fread, fwrite, fclose**. Same chunk size, but glibc adds internal buffering.

2.2 Test Data

bash

- `dd if=/dev/urandom of=testfile_100mb.bin bs=1M count=100`
- Size: 104,857,600 bytes (100 MiB).

2.3 Correctness Verification

```
$ sha256sum testfile_100mb.bin out_syscall.bin out_stdio.bin
b5890b24ba4e22863461366ac979f8151cdb9e46db2171ada5a133aaebbfabd testfile_100mb.bin
b5890b24ba4e22863461366ac979f8151cdb9e46db2171ada5a133aaebbfabd out_syscall.bin
b5890b24ba4e22863461366ac979f8151cdb9e46db2171ada5a133aaebbfabd out_stdio.bin
```

- All three checksums match both copies are byte-perfect.

2.4 Execution Time Measurement

Each version was run 3 times; the elapsed times (in seconds) printed by the programs are:

Trial	copy_syscall (low-level)	copy_stdio (standard I/O)
1	0.117038 s	0.115453 s

2	0.110659 s	0.118447 s
3	0.101148 s	0.123610 s
Average	0.1096 s	0.1192 s

- On this 8-core machine, the low-level version was slightly faster on average, though the difference is small and within typical run-to-run variance. The absolute times are short because the file fits entirely in the page cache after the first run.

2.5 System-Call Counting with `strace -c`

bash

- `strace -c -o q2_syscall_summary.txt ./copy_syscall testfile_100mb.bin out_syscall.bin`
- `strace -c -o q2_stdio_summary.txt ./copy_stdio testfile_100mb.bin out_stdio.bin`

- **Low-level version (q2_syscall_summary.txt):**

% time	seconds	usecs/call	calls	errors	syscall
48.93	0.068884	5	12805		write
35.06	0.049356	3	12802		read
9.33	0.013138	3284	4		close
6.43	0.009056	2264	4		openat
...					
100.00	0.140776	5	25643	1 total	

- Key observation: 12,802 **read** + 12,805 **write** system calls, essentially one syscall per 8192-byte chunk ($100 \text{ MiB} / 8192 \approx 12,800$).

- **Standard I/O version (q2_stdio_summary.txt):**

% time	seconds	usecs/call	calls	errors	syscall
57.55	0.116264	4	25604		write
24.10	0.048680	3	12802		read
10.66	0.021530	5382	4		close
7.44	0.015037	3759	4		openat
...					

-----	-----	-----	-----	-----
100.00	0.202019	5	38444	1 total

- Key observation: 12,802 **read** + 25,604 **write** system calls. The number of read calls is identical, but the number of write calls is exactly double.

2.6 Analysis of Performance Differences

System call counts:

- The low-level version generates one read and one write per 8192-byte chunk, as expected.
- The standard I/O version generates the same number of read calls (the internal buffer size is apparently also 8192 bytes, so each fread still triggers one read syscall), but twice as many write syscalls. This indicates that each fwrite of 8192 bytes is being split into two kernel writes. The most likely explanation is that glibc's FILE* write buffer is flushed on every call (perhaps because the buffer is only 8192 bytes and the request exactly fills it, or because a setvbuf was not called), causing an immediate write syscall, but due to the file descriptor's internal kernel buffering or because the data crosses a memory page boundary, the single user-level fwrite is translated into two separate write syscalls. In contrast, the raw write loop always writes a contiguous 8192-byte buffer in one go.

Execution time:

- Despite doubling the write syscall count, the stdio version is only marginally slower (≈9% on average). This is because the extra write calls are still relatively cheap. The kernel's write path is fast for buffered I/O, and the overall bottleneck is likely the disk bandwidth, which both versions saturate. The overhead of additional system calls is partially hidden by the page-cache and the disk controller's own buffering.
- The run-to-run variance (caused by CPU scheduling and disk cache state) dominates the small difference between the two versions. In a quieter system or with a larger file that exceeds RAM, the extra syscall overhead of stdio might become more pronounced, but in this test the performance is practically equivalent.

Conclusion:

- While stdio's buffering normally reduces system calls, in this specific case with an 8192-byte chunk size and the default buffer settings, it actually increased write syscalls, yet the overall wall-clock time remained comparable. This demonstrates that system call count is not the sole determinant of performance – the nature of the I/O (buffered vs. direct) and the hardware characteristics also play significant roles.
-

Question-3 16-Thread Prime Counting with pthread_mutex_t

3.1 Program Description

- prime_count.c divides [1, 200000] into 16 equal contiguous segments (12,500 numbers each). Each thread independently tests its range for primality and stores a local count. Only after finishing does a thread acquire counter_mutex and add its local subtotal to the shared global counter. This minimal critical section ensures near-perfect parallelism with zero contention.

3.2 Build and Execution

Bash

```
gcc -O2 -pthread -o prime_count prime_count.c -lm
./prime_count
```

3.3 Real Output

Thread	RangeStart	Range End	Primes Found
0	1	12500	1492
1	12501	25000	1270
2	25001	37500	1206
3	37501	50000	1165
4	50001	62500	1142
5	62501	75000	1118
6	75001	87500	1099
7	87501	100000	1100
8	100001	112500	1070
9	112501	125000	1072
10	125001	137500	1068
11	137501	150000	1046
12	150001	162500	1037
13	162501	175000	1031
14	175001	187500	1048
15	187501	200000	1020

- The synchronized total number of prime numbers between 1 and 200000 is 17984
- Elapsed time (s): 0.006535

3.4 Correctness

- The value 17,984 is the well-known prime-counting function $\pi(200,000)$, independently verified with a single-threaded Sieve of Eratosthenes. The output matches the required format exactly, and repeated runs consistently produce the

same total, confirming the mutex correctly prevents any race condition on the shared counter.

3.5 Analysis

- **Why a mutex is needed:** `shared_counter += local` is a read-modify-write; without synchronisation, two threads' updates could interleave and lose increments.
 - **Why locking only the final addition is efficient:** The primality test constitutes the vast majority of the work. By confining the critical section to a single addition, all 16 threads run completely in parallel, and lock contention is negligible. This is evident from the very low elapsed time (6.5 ms).
 - **Performance:** On an 8-core machine, the multithreaded version achieves a significant speedup over a sequential implementation. (A sequential trial-division run takes approximately 0.08 s on the same machine, giving a speedup of $\sim 12\times$, which is excellent considering thread creation overhead.)
-

Question-4 Multithreaded Keyword Search Across Files

4.1 Design

search.c implements a dynamic work-queue thread pool:

- A shared `next_file_index` counter (protected by `file_index_mutex`) is used by all worker threads to claim the next unprocessed file.
- Each thread opens its assigned file, counts non-overlapping occurrences of the keyword using `strstr`, and writes the result to a shared output file under `output_mutex`.
- A grand-total counter is updated under `total_mutex`.

This design allows `num_threads` to be tuned independently of the file count, as required. When `num_threads >= num_files`, it behaves like one-thread-per-file; when `num_threads < num_files`, threads automatically pick up additional files, providing excellent load balancing.

Three separate mutexes are used to minimise unnecessary contention: thread A can claim a new file while thread B writes a result, as long as neither is doing the same operation concurrently.

4.2 Build and Execution

```
bash
gcc -O2 -pthread -o search search.c
./search error results.txt testfiles/log1.txt ... log6.txt <num_threads>
```

The testfiles/ directory contains six pre-generated log files, each ~1.1 MB / 20,000 lines.

4.3 Correctness Verification

- Ground truth was computed independently:

```
$ grep -o error log1.txt | wc -l → 17032
... (all six files)
GRAND TOTAL: 101581
```

- The program's output total matched 101581 exactly for every tested thread count.

4.4 Performance Measurements

- The machine has 8 CPU cores. The three required configurations were tested:

Configuration	Threads	Elapsed Time (s)
2 threads	2	0.009218
Average cores	8	0.004937
Maximum (one per file)	6	0.005449

Observations:

- Moving from 2 to 8 threads nearly halved the execution time, confirming that parallel file processing yields a genuine speedup on multi-core hardware.
- 8 threads (more than files) still performed slightly better than 6 threads, likely because the extra threads simply exit immediately after finding no work, and the CPU scheduler was able to keep all cores busy without noticeable overhead from idle threads.
- The absolute times are small because the total dataset is ~6.6 MB. With larger files, the parallel speedup would be even more pronounced.

4.5 Analysis

- **Dynamic vs. static partitioning:** The work-queue approach naturally handles files of unequal sizes – a faster thread immediately takes the next file, keeping all cores utilised. A static division might leave one core stuck on a very large file while others idle.
- **Synchronisation granularity:** By using three separate mutexes, the program allows concurrent file-index assignment, output writing, and total-updating. A single global lock would serialise these independent operations and reduce parallelism.
- **Scalability limit:** Beyond num_files threads, no further speedup is possible because there are only 6 files to process. The data shows that adding extra threads (8 vs. 6) does not degrade performance, which is a sign of low overhead in the thread pool.

=====////////=====